

METHODOLOGY

3.1 Introduction to the Proposed Architecture.

The CacheFlow system is built to facilitate Mixture-of-Experts (MoE) inference with extreme memory constraints on GPU hardware. The fundamental implementation is deeper than mere GPU-based cache: a novel three-level memory hierarchy where disk-based expert weights are smoothly transferred to staging the pinned CPU RAM at the GPU VRAM slots via non-blocking, asynchronous PCIe transfers. This architecture essentially separates graphics memory demands and the overall count of parameters in an expert model, allowing execution of huge MoE models on hardware with limited resources.

The suggested method is an optimization at the system level, which does not alter the original MoE architecture or the structure of the transformer models. Rather, it runs on the layer level by replacing conventional parallel expert modules with an optimized implementation which has built-in direct disk I/O, smart memory staging, asynchronous prefetching and temporal cache freezing functionality. Instead of storing all expert weights on host RAM (as in traditional solutions), CacheFlow takes advantage of the fact that expert weight files are compactly stored on the disk in the safetensors format. This system directly reads the necessary expert weights on-demand out of disk, stages them via CPU-pinned memory buffers and asynchronously transfers them to GPU slots. An advanced caching scheme that intra-batch freezes keep common experts in GPU memory and cold experts loadable and able to be accommodated with a fixed capacity constraint.

Such a three-tier architecture (Disk, Pinned RAM, and GPU VRAM) forms a memory-performance trade-off space in which capacity can be configured on a per-application basis. The system is computational correct and experiences dramatic memory footprints cuts on the GPU and allows previously unreachable models to be run by resource-constrained systems.

3.2 System Architecture and Memory Pipeline

3.2.1 Three-Tier Memory Hierarchy

The CacheFlow system clearly defines the storage and access of expert weights in three tiers of memory:

Tier 1: Disk (Persistent):

All the expert weight matrices are stored authoritatively.

Expert weights are stored in the safetensor format which is a compressed binary serialization format that allows random access of element.

The access is implemented through MmapExpertStore, which does a zero-copy slicing of expert weight tensors.

Large capacity (model-dependent, usually 10-100 GB in the case of large MoEs)

Low access bandwidth (100-500MB/s -based on storage medium)

None, no GPUs involved: weights are read off disk by CPU.

Tier 2: Pinned Host Memory (Pinned Staging Buffer):

Fixed size collection of CPU-resident memory buffers with page-locking semantics of CUDA.

Used as a temporary loading point of weights in disk.

The weight matrices of each staging buffer are that of one expert.

Pinned allocation enables asynchronous and zero-copy PCIe transfers to GPU without software involvement.

Typically allocated with `torch.cuda.pin_memory()`

A hyperparameter that can be adjusted is number of staging buffers (`staging_slots`) (e.g., 8 or 16).

Provides a decoupling point between the slow disk I/O and the fast GPU transfer.

Tier 3: GPU VRAM(Working Set):

Active computation and inference is reserved.

Has: model static parameters (embeddings, attention layers, output head), intermediate activations, and a fixed set of expert weight slots in GPU memory.

Smaller capacity of between 6 GB to 24GB on consumer grade hardware.

Very high-bandwidth access (up to 900 GB/s on current NVIDIA GPUs)

Expert slots are executed as a preallocated `torch.Tensor` of shape `(numgpulots, outputsize, inputsize)`.

3.2.3 Direct Disk I/O through MmapExpertStore.

The `MmapExpertStore` class is an abstraction of expert weight access to disk storage. In contrast to conventional methods which store complete weight matrices in RAM, `MmapExpertStore` uses a slice-based method, which is memory-efficient:

Initialization Phase:

```
store = MmapExpertStore(weight_file)
```

The caller of `readmetadata` checks the `safetensors` file header to identify:

The tensor key of expert weights (usually, `model.experts.weight`, or similar)

The shape of the tensor (`numexperts, outputsize, inputsize`)

The type of data in the tensors (`float32, float16` or `int8`)

The complexity of this metadata reading is $O(1)$, and it only needs to check the headers; no weight data is read into memory.

Per-Expert Loading:

```
expertweight = store.loadexpert(expertid, out=stagingbuffer)
```

Load expert method, load expert, follows the steps as follows:

Bounds Checking: Check that 0 expertid numexperts.

Safetensors Slicing: Open the safetensors file and call `getslice(safetensors)` with only the weight matrix number of expertid. The slice operation of safetensors retrieves an exact outputsize by inputsize by dtype_bytes bytes of data on disk, ignoring the data of previous experts.

Buffer Population: Transfer the sliced weights to the given out buffer (a pinned memory allocation). This copy operation involves effective CPU memory operations but does not send data to GPU at this point.

Return: Publish the filled-in buffer tensor.

Its major strength lies in the fact that disk I/O is proportional to the size of a single expert, and not the weight file as a whole. On a model with 60 100 MB experts (6 GB in total) loading a single expert requires only 100 MB of disk.

3.2.3 Staged Pinned Memory and Asynchronous PCIe Transfer.

In GraniteMoeParallelExperts the nextstaging() method is used to control the lifecycle of pinned memory buffers:

```
def nextstaging(self) -> Optional[torch.Tensor]:
```

Gets a free pinned buffer, or blocks until one is free.

Once the disk I/O of the buffer is made, it is immediately scheduled to.

asynchronous transfer to graphics card. The system is awaiting previous transfers when necessary.

Staging pipeline works in the following manner:

Buffer Availability Check: Have a list of pinned buffers. When buffers are in-flight (waiting PCIe transfers), the system blocks till it finishes.

Synchronization Point: It is important to be sure that any previous PCIe transfer that used the same buffer has completed before reusing it. This is monitored through torch.cuda.Event objects.

Return Buffer: Provide a ready pinned buffer to be used in the next disk read operation.

Once a disk weight has been loaded into a pinned buffer, a scheduled asynchronous PCIe transfer is scheduled using torch.cuda.Stream:

```
stream = torch.cuda.Stream()
```

```
with torch.cuda.stream(stream):
```

```
    gpuslot[:] = pinnedbuffer # DMA through PCIe.
```

```
    event = torch.cuda.Event()
```

```
event.record(stream) # Record completion in stream.  
pending[expertid] = (event, gpuslot) # Follow through pending transfer.
```

This design enables loading of the weights of multiple experts on disk and transmitting them to the GPU simultaneously. As the CPU is reading the weights out of disk, the weights of Expert B that were previously loaded into the graphics card via the PCIe process are being sent to the graphics card, and the computation of Expert C is being performed on the graphics card. Overall latency is dramatically reduced by this overlapping of I/O, transfer and computation.

4.5 ExpertCache: Freezed Cache on GPU Residents.

The ExpertCache class handles a fixed size LRU cache of experts in GPU memory. In addition to typical LRU operations, it also includes a critical freezing environment to eliminate intra-batch conflicts:

Cache State:

cache: OrderedDict[int, torch.Tensor]: Converts expert IDs into their associated weight tensors on the GPU.

_frozen: set[int]: Set of expert IDs which may not be evicted (locked out in current batch processing)

capacity: int: The maximum number of experts that can be kept in the cache at a time.

Freezing Mechanism:

```
def freeze many(self, expertids: List[int]) none: self._frozen.update(expert_ids)
```

```
def unfreeze many(self, expert id: List[int]) =None: where expert-id correspondingly  
matches the expert-id in expert-ids: self._frozen.discard(expert_id)
```

When forward passing, the list of experts required (by token routing) is frozen in the cache at the start of the pass. This guarantees that when the system needs to fetch an expert which is not in the cache (a cold expert), it does not mistakenly evict a needed expert

which is in the cache. Freezing is set free at the completion of the batch and normal LRU eviction is carried out on the succeeding batch.

[Insert Fig 1: System Memory Hierarchy Diagram Here].

Figure Caption (Figure 1): The CacheFlow memory hierarchy (three levels) represents the entire data flow between persistent disk storage and pinned CPU memory staging (staging) into GPU VRAM. Expert weights are stored in safetensors form in a compact disk. The MmapExpertStore accesses particular slices of experts on demand. The weights are staged in the pinned memory buffers to be transferred asynchronously. The working set of experts is held by the GPU cache. The diagram indicates the presence of a variety of experts in various stages: some of them could be resident in GPU cache, others awaiting transfer to GPU, and others could be loaded to disk.

3.3 CacheFlow Routing Mathematically Modeled.

3.3.1 Memory Budget Decomposition using Three-Tier Architecture.

The per-user GPU memory usage of an optimized model of MoE is partitioned into the three-level hierarchy:

$$M_{\text{VRAM}} = M_{\text{static}} + M_{\text{cache}} + M_{\text{KV}} + M_{\text{act}}$$

Where:

M_{static} : Memory used by the model parameters (embeddings, attention layers, output head, layer normalization) that do not change and are stored in the GPU memory permanently.

$M_{\text{transfer cache}}$: Memory used by expert weights in the VRAM of the graphics card (the working set)

M_{KV} : Cache memory of key-values in attention (where needed)

M_{act} : Memory of intermediate activations in forward computation.

The key innovation here is that the critical innovation is at the level where the parameter of the size of the cache does not depend on the overall number of experts. The cache memory is controlled by:

$$M_{\text{cache}} = L_{\text{MoE}} \times C \times P_{\text{expert}} \times b_o$$

Where:

L_{MoE} = MoE layers in the model.

C : Capacity constraint - the largest number of expert slots in GPU VRAM per layer (i.e. $C = 4$ implies that at any time at most 4 experts can be resident in the GPU VRAM on each layer)

P_{expert} : The size of single expert weight matrix (in terms of number of parameters).

On parameter size (float typed): The size of a parameter (usually 2 bytes (float16) or 4 bytes (float32))

Host Pinned Staging Memory Footprint 3.3.2.

The staged buffer of pinned memory is unique to both the GPU memory and disk storage:

$$M_{Pinned} = L_{MoE} \times S_{staging} \times P_{expert} \times b_o$$

Where:

$S_{staging}$ The quantity of staging buffers (e.g., $S_{staging} = 8$)

More importantly, $S_{staging}$ is often significantly less than the number of experts (S_{slots}). While S_{slots} may be 60 or more, $S_{staging}$ is often 4–16. With this design the system is capable of pipelining disk I/O and PCIe transfers without proportional allocation of pinned memory.

3.3.3 Disk storage and the entire memory budget.

The disk storage is limited and proportional to the number of the overall experts:

$$M_{Disk} = L_{MoE} \times S_{slots} \times P_{expert} \times b_o$$

Where:

S_{slots} : The number of experts in the entire MoE layer (e.g., 60 in the Qwen2-MoE model)

This is done to complete the three-tier budget:

$$M_{Total} = M_{VRAM} + M_{Pinned} + M_{Disk}$$

The critical decoupling is observed:

VRAM is only dependent on C (GPU capacity constraint).

The value of M_{Pinned} only depends on the number of staging buffers (S staging).

The disk size is based on the number of slots on the disk (S_{slots}), and is not limited by hardware.

3.3.4 Memory Reduction Formula

CacheFlow supports the implementation of large expert models on GPUs with memory constraints: by setting capacity constraints C and staging buffers S (staging), it is possible to deploy large expert models on constrained GPUs:

$$\text{VRAM Reduction} = 1 - \frac{C}{S_{\text{slots}}}$$

For example, with $C = 4$ and $S_{\text{slots}} = 60$:

$$\text{VRAM Reduction} = 1 - \frac{4}{60} = 93.3\%$$

This is a 93.3% cut in the amount of expert-related GPU memory usage over loading all experts at once.

3.3.6 Dynamic Cache Buffer Implementation.

The CacheFlow dynamic buffer assigner can be mathematically formulated as:

$$M_{\text{cache}} = L_{\text{MoE}} \times C \times P_{\text{expert}} \times b_o$$

The footprint of host RAM pinning is:

$$M_{\text{Pinned}} = L_{\text{MoE}} \times S_{\text{staging}} \times P_{\text{expert}} \times b_o$$

These equations alone demonstrate how the system can be decoupled between the overall number of parameters and the VRAM requirement through the control of the capacity C .

3.3.6 Token Routing and Hit Efficiency of Caches.

The MoE routing algorithm is used to select the experts to process a token. Gating network makes a distribution of all experts given a single input token. The routing usually picks the K experts using each token (e.g. $K = 2$) according to gating logits.

The team of experts that will be needed in a batch is set as:

$$\text{Required_Experts} = \{i : \text{expert_size}[i] > 0\}$$

The productivity of CacheFlow is directly proportional to the overlap between the needed professionals and the capacity of the cache:

The Cache Hit Rate = $(|\text{cache}/\text{hit}|) = \frac{(\text{t_required experts}) \text{ intersecting with } (\text{t_cached experts})}{\text{t_required experts}}$.

Expert access patterns are the temporal locality, where recent tokens prefer to leave a path to similar sets of experts and this is used to achieve the highest hit rates with a given capacity C .

3.4 Asynchronous Execution Flow

3.4.1 Forward Pass Initialization and Batch Set up.

When a forward pass is called on a GraniteMoeParallelExperts module the following initialization sequence is executed:

Routing Distribution Computation: The gating network uses the input tokens to compute expert routing logits, and generates a distribution over experts.

Identification of Experts needed: The list of experts needed by the current batch is identified:

`required_experts = [i in i,size in enumerate(expert_size)! size > 0].`

The input is divided into parts (input tensors) in this operation through expert assignment to produce sub tensors per expert.

Cache Freezing: The cache is pinned down with all the necessary professionals lest they be evicted by accident:

`cache.freeze_many(required_experts)`

This is so that as the cold experts are fetched, the system will not evict any needed experts that are already wrapped up.

Residency Classification: The system partitions required experts into two classes:

Cached Experts (Cache Hits):

```
cached_experts = [e for e in required_experts if e in cache.cache]
```

Cold Experts (Cache Misses):

```
cold_experts = [e for e in required_experts if e not in cache.cache]
```

3.4.2 Prefetching and pending transfers Asynchronous.

To cold specialists, the system starts asynchronous loading:

```
pending = {} # expert_id -> (torch.cuda.Event, gpu_slot)
```

The following pipeline is launched to each cold expert:

Disk Read (CPU): Read in a staging buffer that is available and load the weights of the expert on disk:

```
staging_buffer = next staging() # Block when all buffers have been started.  
store.load_expert(expert_id, out=staging_buffer)
```

The call load expert (/) reads the expert slice on disk with safetensors, and blocks the CPU thread until the read is finished.

Asynchronous Transfer Scheduling (GPU): Schedule non-blocking PCIe transfer of pinned memory to GPU:

```
stream = torch.cuda.Stream() with torch.cuda.stream(stream): gpu_slot[:] = staging_buffer  
# Non-blocking transfer to GPU slot event = torch.cuda.Event() event.record(stream) #  
Record completion event. pending[expert_id] = (event, gpu_slot)
```

The point to note is that `gpu_slot[:] = staging_buffer` is asynchronous; the DMA engine of the GPU is involved in the transfer without blocking either the CPU or the GPU compute. The system does not wait till it is completed.

Overlap with Computation: As the weights of cold experts are being transferred:

At the GPU, the computations of the forward pass of the cache experts commences.

Other weight weights of cold experts may have their disk reads started by the CPU.

There are several transfers of multiple experts simultaneously through PCIe.

3.4.3 Before Expert Computation Synchronization.

The system should make sure that the weights of a cold expert have been loaded into GPU memory even before calculating its contribution:

```
in case expert-id in pending: event, gpu_slot = pending[expert_id] event.synchronize() #  
Wait until PCIe transfer has completed. out_chunks[expert_id] =  
F.linear(input_list[expert_id], gpu_slot) del pending[expert_id] else:
```

Expert already was etched; calculate straight off.

```
expert_weight = cache.cache[expert_id] out_chunks[expert_id] =  
F.linear(input_list[expert_id], expert_weight)
```

The event.synchronize call prevents the CPU from running until the DMA transfer of the GPU is complete. This makes sure that weights of the expert are in GPU memory before computation can commence.

1.3.3.4.4 Execution Ordering to reduce Latency.

The execution order will be selected in such a way that wait time will be minimized:

```
execution_order = cached_experts + cold_experts
```

This order guarantees that:

Expert cache requests are done instantly (no delayed transfer).

Cold experts would be processed in sequence once their transfers have been completed asynchronously.

When the system gets to a cold expert, all possible time has passed to get the weights off disk to staging to GPU.

[INSERT FIGURE 2: Asynchronous Execution Flow and Prefetching Diagram Here]

Figure Caption (Figure 2): The diagram (asynchronous execution flow) shows how the disk I/O, PCIe transfer, and the GPU computation overlap. At Time T0 Expert A is being read out of disk, Expert B is being transferred over PCIe, and Expert C is being calculated on the GPU. In-flight transfers are recorded by the pending dictionary. Freezing of caches safeguards the needed experts. The illustration depicts the time-based development of different specialists along the pipeline: Disk I/O → Pinned Staging → PCIe Transfer → GPU Computation.

3.6.2 Group Interaction and Unfreezing of the Cache Memory.

After processing all the required experts:

```
output = torch.cat(out_chunks, dim=0) cache.unfreeze_many(required_experts)
```

Expert output is in a different order, so to rebuild the output tensor, the concatenated expert outputs are in the original order of the batch. The release of cache freezing can occur, and normal LRU eviction can take place with the next batch.

3.5 More Advanced LRU Cache Management and Freezing.

3.5.1 Freezing with Cache state representation.

ExpertCache class is a freezing extension of of standard LRU caching:

```
cache: OrderedDict[int, torch.Tensor] # expert_id -> gpu_weight frozen: set[int] #  
expert_ids which cannot be evicted this round.
```

The order of insertion and access is preserved in the OrderedDict. When an expert is visited the key is moved at the end of the file indicating it as the most recently accessed. The first candidate on the order is the least recently used candidate to be evicted.

3.5.2 Cache Hit Path

In case of a call to `get(expert_id)` and the expert is in the cache:

```
def get(self, expert_id: int) Optional[torch.Tensor]: in case expert id not in self.cache:  
return None self.cache.move_to_end(expert_id) return self.cache[expert_id]
```

This operation:

Existence checks in $O(1)$ time.

Moves the expert to the end of the OrderedDict (mark as recently used)

Brings back the weight tensor in the cache with no data transfer.

Hits in the cache do not have any PCIe transfer or disk I/O.

3.5.3 Cache Miss, available free slots.

In case `put(expert_id, gpu_weight)` gets invoked and there exists free cache space:

```
if len(self.cache) < self.capacity: self.cache[expert_id] = gpu_weight
```

The weight tensor is just appended in the cache. The new expert will be the most recent used one.

3.5.4 Cache Miss Full Capacity (Eviction)

Upon this operation when the cache is full and a new expert is needed, the least recently used unfrozen expert is kicked out:

```
if len(self.cache) >= self.capacity: Make effort to evict unfrozen experts in LRU order. is  
victim in self.cache.keys list(self.cache.keys): when not victim_id in self._frozen: del  
self.cache[victim_id] break
```

when all the frozen experts are frozen, block (rare case)

```
if len(self.cache) >= self.capacity: throw RuntimeError ( Cache full and all experts frozen )
```

```
self.cache[expert_id] = gpu_weight
```

Critical Detail - Freezing Protection: The loop is repeated in LRU sequence (least recently used to most recently used). It jumps any frozen (`_frozen` set) expert. Candidates to eviction are only unfrozen experts.

This structure guarantees that, in a batch, no necessary specialist is evicted by accident. This is ensured by the freezing mechanism wherein in case Expert 5 is needed by the batch

at hand (and thus cacheable or in the process of being cacheable), it cannot be evicted to accommodate Expert 4 even in the event that Expert 5 is not as actively used as of late.

After an expert has been kicked out of the GPU cache:

It has its weight tensor offloaded out of the GPU memory.

It is deleted out of the cache dictionary.

In the event that the expert is required in a subsequent batch it will be read back in (cache miss).

The weight information of the expelled expert is safely on disk; a writeback is not required.

3.5.5 Full Cache Lifecycle in Forward Pass.

The entire process of the lifecycle of a single forward pass is:

Freeze Phase: All the specialists are frozen to initiate any transfers.

Fetch Phase: Cold masters are brought in out of band. The cache can expel unfrozen experts (experts not needed by the current batch) to accommodate them.

Compute Phase: Experts are computed sequentially. Cached professionals will immediately compute; cold professionals will wait until their asynchronous transfers are done.

Unfreeze Phase: By the time the next batch is complete the unfreezing of all the professionals is made and in this case, they can be evicted unless the next batch needs them.

This is repeated every forward pass (each token or batch of tokens in generation).

[INSERT FIGURE 3: Advanced LRU Cache and Freezing Mechanism Diagram Here]

Figure Caption (Figure 3): The advanced LRU cache with freezing mechanism shows how intra-batch protection works. Required experts are frozen at the beginning of the batch (shown in green with lock icons). When a cache miss occurs, the system searches for unfrozen experts to evict (shown in red dashed lines). Frozen experts cannot be evicted, ensuring their protection. The diagram shows a cache state where Expert 2 and Expert 9

are frozen (required by current batch), while Expert 5 is unfrozen and thus is the candidate for eviction when Expert 4 is loaded.

3.6 Implementation Environment

3.6.1 Software Stack

The Python 3.8+ CacheFlow implementation will use the following core libraries:

PyTorch (1.13+): PyTorch offers operations on tensors, gpu memory handling, cuda stream and event control, and supports all the computations of neural networks. Specifically, `torch.cuda.Stream()` and `torch.cuda.Asynchronous` transfer coordination is done by using `Event()`s.

Safetensors (0.3.1+): Implements the safe open context manager and the `get_slice` method to extract expert weight slices efficiently and without any copying of data to the storage, as well as to disk-resident weight files.

Transformers Library (Hugging Face, 4.0+): Allows loading of pretrained MoE model settings (e.g., IBM Granite, Qwen2-MoE), and tokenization utility.

CUDA Toolkit (11.8+): It offers the capability of utilizing the GPU compute, memory, and PCIe DMA.

Accelerate Library (0.20+): Makes it easy to use several devices as inference and model loading strategies.

3.6.2 Hardware Configuration

The implementation is tested on the hardware setups in the following configurations:

The environment of Development and Experimentation:

GPU: NVIDIA T4 (16 GB VRAM) or NVIDIA H100 (80 GB VRAM) in the cloud environments (Google Colab, Lambda Labs)

CPU: x86-64 processor with 16-64 GB system RAM, PCIe to graphics card.

Storage: NVMe SSD (to access the disk fast, which allows to run `MmapExpertStore` operations efficiently) or network-attached storage.

Interconnect: PCIe 4.0 or PCIe 3.0, which offers a theoretic bandwidth of 1632 GB/s in case of transfers of the GPU.

Target Deployment Hardware:

Consumer-grade GPUs: NVIDIA RTX 3060 (12 GB), RTX 4070 (12 GB), RTX 4090 (24 GB)

Embedded Systems: NVIDIA Jetson Orin (12 GB VRAM, share with CPU)

Storage: External USB 3.1 or local SSD (500 MB/s read bandwidth will be required at minimum)

System RAM: 32-64 GB minimum amount of pinned memory allocation and staging buffers.

3.6.3 Model Under Test

The analysis is centered on the models that have the GraniteMoeParallelExperts architecture. To evaluate this work, primary is used:

IBM Granite 3.1-3B-A800M-Instruct:

Architecture: Transformer-based, sparse MoE routing in particular layers.

Total Parameters: 3 billion or so.

MoE Structure: Multiple layers of MoE each with 60 experts.

Expert Dimensions: Generally 2048 with 8192 intermediate dimension hidden dimensions.

Expert Floating-point: float16 (half precision) or int8 (quantized).

Mode of inference: Causal language prediction (next-token prediction)

Weight File Format: safetensors (significantly compressed, slice-accessible)

This model class selection enables a controlled benchmarking on real, production scale MoE architectures but can be available to undergraduate research projects.

3.6.4 Experimental Protocol

CacheFlow validation is structured in the following way:

Baseline Measurement: Fill in the naive expert loading the model (all experts in GPU memory or standard disk-loading without staging). Measure:

Maximum memory (in GB) usage by GPUs (through `torch.cuda.max_memory_allocated()`)

Possibly the most important: Inference latency (time per token generated averaged over 50 tokens).

Throughput (tokens/s)

Out-of-memory errors (where applicable)

Configuration Space Exploration: Experiment with the CacheFlow system having different capacity constraints:

GPU Slot Capacity: $C \in \{2, 4, 8, 16\}$

Staging Buffers: $S_{\text{staging}} \in \{4, 8, 16\}$

Post-Optimization Measurement: Measure: Each configuration:

Maximum GPU memory usage (through `torch.cuda.max_memory_used`)

Latency in inference (time per token)

Throughput (in tokens/s)

Hits to cache (number of experts hit / number of expert requests)

Evictions (number of times a disk has to be reloaded by the cache)

disk I/O time (time spent in `store.load expert`)

PCIe transfer time (time spent in `event.synchronize()` waits)

Inference Tasks:

Short: Generate 20 new tokens on the prompt Briefly explain how Mixture-of-Experts models work.

Long-form: Use 100 new tokens on the following prompt: "Write an essay on the architecture of normal Transformers and Mixture-of-Experts models in detail!"

These tasks enable one to observe the behavior of the cache in both short and long sequences.

Metrics Logging: All metrics are recorded with:

CUDA profiling APIs of PyTorch: `torch.cuda.max_memory_allocated`, `torch.cuda.synchronize`.

Specialized timing instrumentation around `store.load_expert`, and `store.next-staging`, and `event.synchronize` invocations.

ExpertCache cache (hit/miss counts, frozen set size) statistics.

3.6.5 Correctness Validation

Numerical accuracy is achieved by running the optimized model on the input logits of the reference baseline:

Deterministic Execution: Both the baseline and CacheFlow-optimized models are run in a deterministic mode (where possible disabling non-deterministic operations in CUDA) with fixed input prompt and random seed.

Token-Level Comparison: The sequence of output tokens is compared to get an exact match, and thus:

The decision of routing is the same (the same experts are chosen when working with each token).

Professional calculations can give the same outcomes.

Output logits are different only at the floating-point roundoff errors.

Numerical Tolerance: Logit-level numerical accuracy is checked to be within floating-point tolerance:

Float32: Relative error $< 1e-5$.

Float16: Relative error of less than $1e-3$.

Inference Stability: The system is run on 10 distinct input prompts to confirm the accuracy of the system when the routing patterns are different.

This verification shows that, the CacheFlow scheduling mechanism disk loading, staging, asynchronous transfer, freezing and eviction does not create any computational errors or affect the inference behavior of the model.

Summary

The CacheFlow approach offers a system level framework to support an efficient MoE inference on memory constrained GPUs with a new three-level memory hierarchy. The system relieves the dependency of the number of expert parameters in total on the count of parameters, staging piping with pinned memory, and asynchronous data transfer via PCIe decouples the memory needs of the GPUs. The cache freezing introduction guarantees accuracy in dynamic, batch-contingent situations where the necessary experts have to be safeguarded against eviction by accident. The mathematical description explains how capacity limits regulate the consumption of the memory in the GPUs regardless of the overall number of experts. The overlapping of disk I/O, PCIe transfer, and GPU computation are shown by asynchronous execution flow to reduce latency. State-of-the-art LRU cache management including intra-batch freezing is used to guarantee efficient utilization of the cache, whilst guaranteeing safety. It is based on production intent MoE architectures and tested on realistic hardware setups and has been shown to be practically applicable to implementing large MoE models on resource constrained systems that could otherwise not support such models.